

Modelo Cobros - Documentación

Descripción General

El modelo **Cobros** gestiona el sistema de cobranza de la aplicación Talentus, permitiendo administrar los pagos recurrentes asociados a clientes, vehículos y contratos. Incluye control de vencimientos, estados, y relación con múltiples entidades del sistema.

Información Técnica

Namespace

```
App\Models\Cobros
```

Tabla de Base de Datos

```
cobros
```

Traits Implementados

- **HasFactory** - Permite crear factories para testing
- **SoftDeletes** - Implementa eliminación suave (soft delete)
- **LogsActivity** (Spatie) - Registra automáticamente cambios en el modelo

Observer

```
#[ObservedBy(CobrosObserver::class)]
```

Utiliza el observer **CobrosObserver** para lógica automática en eventos del modelo.

Global Scope

Implementa **EmpresaScope** para filtrado automático por empresa (multi-empresa).

Estructura de la Tabla

Según la imagen de phpMyAdmin proporcionada, la tabla **cobros** tiene la siguiente estructura:

Campo	Tipo	Nulo	Predeterminado	Descripción
-------	------	------	----------------	-------------

Campo	Tipo	Nulo	Predeterminado	Descripción
id	bigint	No	AUTO_INCREMENT	Identificador único
clientes_id	bigint	No	-	FK a tabla clientes
vehiculos_id	bigint	Sí	NULL	FK a tabla vehiculos (opcional)
producto_id	bigint	Sí	NULL	FK a tabla productos (opcional)
contratos_id	bigint	Sí	NULL	FK a tabla contratos (opcional)
comentario	text	Sí	NULL	Comentarios adicionales
periodo	varchar(191)	No	-	Periodo de cobro (mensual, trimestral, etc.)
divisa	varchar(191)	No	-	Moneda (PEN/USD)
monto_unidad	decimal(10,2)	Sí	NULL	Monto por unidad/vehículo
cantidad_unidades	int	Sí	NULL	Número de unidades a cobrar
tipo_pago	varchar(191)	No	-	Tipo de pago (FACTURA/RECIBO/etc.)
nota	varchar(191)	Sí	NULL	Notas adicionales
observacion	varchar(191)	Sí	NULL	Observaciones
fecha_inicio	date	Sí	NULL	Fecha de inicio del cobro
fecha_vencimiento	date	No	-	Fecha de vencimiento
estado	enum('0','1','2')	No	0	Estado del cobro
suspendido	tinyint(1)	No	0	Indica si está suspendido
empresa_id	bigint	Sí	NULL	FK a tabla empresas (multi-empresa)
created_at	timestamp	Sí	NULL	Fecha de creación
updated_at	timestamp	Sí	NULL	Fecha de última actualización
deleted_at	timestamp	Sí	NULL	Fecha de eliminación suave

Propiedades y Configuración

Fillable/Guarded

```
protected $guarded = ['id', 'created_at', 'updated_at'];
```

Todos los campos son asignables masivamente excepto `id`, `created_at` y `updated_at`.

Cast

```
protected $casts = [  
    'clientes_id' => 'integer',  
    'vehiculos_id' => 'integer',  
    'contratos_id' => 'integer',  
    'fecha_inicio' => 'date:Y-m-d',  
    'fecha_vencimiento' => 'date:Y-m-d',  
    'vencido' => 'boolean',  
];
```

Constantes

```
public const VENCIDOS = '2';
```

Representa el estado de cobros vencidos.

Activity Log

```
protected static $recordEvents = ['deleted', 'created', 'updated'];
```

Registra eventos de creación, actualización y eliminación.

```
public function getActivitylogOptions(): LogOptions  
{  
    return LogOptions::defaults()  
        ->logUnguarded()  
        ->logOnlyDirty();  
}
```

Solo registra campos que han cambiado realmente.

Relaciones Eloquent

Relaciones BelongsTo (Muchos a Uno)

clientes()

```
public function clientes()  
{  
    return $this->belongsTo(Clientes::class, 'clientes_id')->withTrashed();  
}
```

- **Relación:** Un cobro pertenece a un cliente
- **FK:** `clientes_id`
- **Incluye registros eliminados:** Sí (`withTrashed()`)

vehiculo()

```
public function vehiculo()  
{  
    return $this->belongsTo(Vehiculos::class, 'vehiculos_id')->withTrashed();  
}
```

- **Relación:** Un cobro puede estar asociado a un vehículo específico
- **FK:** `vehiculos_id`
- **Incluye registros eliminados:** Sí (`withTrashed()`)
- **Opcional:** Sí (nullable en BD)

contrato()

```
public function contrato()  
{  
    return $this->belongsTo(Contratos::class, 'contratos_id');  
}
```

- **Relación:** Un cobro puede estar vinculado a un contrato
- **FK:** `contratos_id`
- **Opcional:** Sí (nullable en BD)

producto()

```
public function producto()  
{  
    return $this->belongsTo(Productos::class, 'producto_id');  
}
```

- **Relación:** Un cobro puede estar asociado a un producto/servicio
- **FK:** `producto_id`
- **Opcional:** Sí (nullable en BD)

Relaciones HasMany (Uno a Muchos)

detalle()

```
public function detalle()
{
    return $this->hasMany(DetalleCobros::class, 'cobros_id');
}
```

- **Relación:** Un cobro puede tener múltiples detalles (vehículos incluidos)
- **Modelo relacionado:** `DetalleCobros`
- **FK en tabla detalle:** `cobros_id`
- **Uso:** Gestiona los ítems individuales del cobro por vehículo

payments()

```
public function payments()
{
    return $this->hasMany(Payments::class, 'cobros_id');
}
```

- **Relación:** Un cobro puede tener múltiples pagos
- **Modelo relacionado:** `Payments`
- **FK en tabla payments:** `cobros_id`
- **Uso:** Registra todos los pagos realizados contra este cobro

Scopes (Query Scopes)

scopeVencido()

```
public function scopeVencido($query, $estado = NULL)
{
    return $query->where('vencido', $estado);
}
```

Uso: Filtrar cobros por estado de vencimiento

```
Cobros::vencido(true)->get(); // Solo vencidos
Cobros::vencido(false)->get(); // Solo no vencidos
```

scopeEstado()

```
public function scopeEstado($query, $estado = NULL)
{
    return $query->where('estado', $estado);
}
```

Uso: Filtrar cobros por estado

```
Cobros::estado('1')->get(); // Solo activos
Cobros::estado('0')->get(); // Solo inactivos
```

scopeStatus()

```
public function scopeStatus($query, $estado = NULL)
{
    return $query->orWhere('estado', $estado);
}
```

Uso: Agregar condición OR para filtrar por estado

```
Cobros::where('campo', 'valor')->status('1')->get();
```

scopeVerificado()

```
public function scopeVerificado($query, $estado = 0)
{
    return $query->where('verificado', $estado);
}
```

Uso: Filtrar por estado de verificación

```
Cobros::verificado(1)->get(); // Solo verificados
```

Métodos Estáticos

createItems()

```
public static function createItems(Cobros $cobro, $cobroItems, $type = 'create')
{
    foreach ($cobroItems as $cobroItem) {
        $cobroItem['cobros_id'] = $cobro->id;
        $cobro->détailé()->create($cobroItem);
    }

    return $cobro->ventaDetalles;
}
```

Propósito: Crear múltiples detalles de cobro de forma masiva

Parámetros:

- **\$cobro** (Cobros): Instancia del cobro padre
- **\$cobroItems** (array): Array de items a crear
- **\$type** (string): Tipo de operación ('create' por defecto)

Retorna: Colección de detalles creados

Ejemplo de uso:

```
$cobro = Cobros::create([...]);
$items = [
    ['vehiculos_id' => 1, 'fecha' => '2024-01-15', 'monto' => 150.00],
    ['vehiculos_id' => 2, 'fecha' => '2024-01-15', 'monto' => 150.00],
];
Cobros::createItems($cobro, $items);
```

Flujo de Trabajo Típico

1. Creación de un Cobro

```
$cobro = Cobros::create([
    'clientes_id' => 1,
    'periodo' => 'MENSUAL',
    'divisa' => 'PEN',
    'monto_unidad' => 150.00,
    'cantidad_unidades' => 5,
    'tipo_pago' => 'FACTURA',
    'fecha_inicio' => '2024-01-01',
]);
```

```
'fecha_vencimiento' => '2024-01-31',  
'estado' => '1',  
]);
```

2. Agregar Detalles (Vehículos)

```
$detalle = [  
    ['vehiculos_id' => 1, 'fecha' => '2024-01-31', 'monto' => 150.00, 'estado'  
=> 1],  
    ['vehiculos_id' => 2, 'fecha' => '2024-01-31', 'monto' => 150.00, 'estado'  
=> 1],  
];  
Cobros::createItems($cobro, $detalle);
```

3. Registrar Pagos

```
$cobro->payments()->create([  
    'fecha_pago' => now(),  
    'monto' => 300.00,  
    'metodo_pago' => 'TRANSFERENCIA',  
]);
```

4. Consultar Cobros Vencidos

```
$vencidos = Cobros::vencido(true)  
->estado('1')  
->with(['clientes', 'detalle.vehiculo'])  
->get();
```

5. Consultar Cobros por Cliente

```
$cobrosCliente = Cobros::where('clientes_id', $clienteId)  
->with(['detalle', 'payments'])  
->orderBy('fecha_vencimiento', 'desc')  
->get();
```

Estados del Cobro

Según el enum en la base de datos:

Valor	Descripción
'0'	Inactivo/Borrador
'1'	Activo
'2'	Vencido (constante VENCIDOS)

Integración con Otros Módulos

Controlador

- **CobrosController**: CRUD básico de cobros
 - `index()`: Listado con filtros
 - `create()`: Formulario de creación
 - `show()`: Ver detalle de cobro
 - `edit()`: Editar cobro existente

Componentes Livewire

- `Admin\Cobros\Index`: Tabla principal con filtros avanzados
- `Admin\Cobros\Save`: Formulario de creación
- `Admin\Cobros>Edit`: Formulario de edición
- `Admin\Cobros>Show`: Vista detallada del cobro
- `Admin\Cobros>Delete`: Eliminación de cobros
- `Admin\Cobros>CreateInvoice`: Generación de facturas desde cobros
- `Admin\Cobros\Payment`: Registro de pagos
- `Admin\Cobros\ModalSuspend`: Suspende cobros
- `Admin\Cobros\ModalActivar`: Activar cobros

Rutas Principales

```
Route::get('cobros', [CobrosController::class, 'index'])->
    name('admin.cobros.index');
Route::get('cobros/crear', [CobrosController::class, 'create'])->
    name('admin.cobros.create');
Route::get('cobros/{cobro}', [CobrosController::class, 'show'])->
    name('admin.cobros.show');
Route::get('cobros/{cobro}/editar', [CobrosController::class, 'edit'])->
    name('admin.cobros.edit');
```

Permisos Relacionados

- `admin.cobros.index` - Ver listado de cobros
- `admin.cobros.create` - Crear nuevos cobros
- `admin.cobros.edit` - Editar cobros existentes

- `admin.cobros.delete` - Eliminar cobros
 - `admin.cobros.exportar` - Exportar datos de cobros
-

Consideraciones Importantes

Multi-empresa

- El modelo usa `EmpresaScope` global, todos los registros se filtran automáticamente por `empresa_id`
- Al crear cobros, asegurar que `empresa_id` esté presente

Soft Deletes

- Los registros eliminados no se borran físicamente
- Usar `withTrashed()` para incluir eliminados en consultas
- Usar `onlyTrashed()` para solo registros eliminados

Activity Logs

- Todos los cambios se registran automáticamente en la tabla `activity_log`
- Solo se registran campos que cambian (`logOnlyDirty()`)

Observer

- `CobrosObserver` puede tener lógica adicional en eventos:
 - `creating()` - Antes de crear
 - `created()` - Después de crear
 - `updating()` - Antes de actualizar
 - `updated()` - Después de actualizar
 - `deleting()` - Antes de eliminar
 - `deleted()` - Después de eliminar

Relaciones Cargadas

Para optimizar consultas, usar eager loading:

```
Cobros::with([
    'clientes.contactos',
    'detalle.vehiculo',
    'payments',
    'producto'
])->get();
```

Filtros Comunes en el Index

Según `Admin\Cobros\Index.php`:

Por Búsqueda

```
->where('razon_social', 'like', '%busqueda%')
->orWhere('placa', 'like', '%busqueda%')
->orWhere('tipo_pago', 'like', '%busqueda%')
```

Por Fecha

- **por_vencer**: Próximos 7 días
- **vencidos**: Fecha <= hoy
- **proximo_mes**: Próximos 30 días

Por Estado

```
->where('estado', $estado)
```

Ejemplos de Uso Avanzado

Cobros con Múltiples Vehículos

```
$cobro = Cobros::with('detalle.vehiculo')->find($id);

foreach ($cobro->detalle as $detalle) {
    echo $detalle->vehiculo->placa;
    echo $detalle->monto;
    echo $detalle->estado ? 'Pagado' : 'Pendiente';
}
```

Total de Pagos Realizados

```
$cobro = Cobros::with('payments')->find($id);
$totalPagado = $cobro->payments->sum('monto');
```

Cobros Vencidos del Mes

```
$vencidos = Cobros::vencido(true)
    ->whereMonth('fecha_vencimiento', now()->month)
    ->with(['clientes', 'detalle'])
    ->get();
```

Generar Factura desde Cobro

```
// Ver componente CreateInvoice.php
$factura = VentasFacturas::generarDesdeCobro($cobro);
```

Campos Calculados / Accessors Potenciales

Aunque no están definidos actualmente, estos serían útiles:

```
// Total del cobro
public function getTotalAttribute()
{
    return $this->monto_unidad * $this->cantidad_unidades;
}

// Saldo pendiente
public function getSaldoPendienteAttribute()
{
    return $this->total - $this->payments->sum('monto');
}

// Está vencido
public function getEstaVencidoAttribute()
{
    return $this->fecha_vencimiento < now();
}

// Días de vencimiento
public function getDiasVencimientoAttribute()
{
    return now()->diffInDays($this->fecha_vencimiento, false);
}
```

Testing

Factory

```
Cobros::factory()->count(10)->create();
```

Tests Recomendados

- Creación de cobros
- Asociación con detalles

- Cálculo de totales
 - Registro de pagos
 - Filtrado por estados
 - Soft delete
-

Changelog y Mejoras Sugeridas

Mejoras Potenciales

1. Agregar accessors para campos calculados (total, saldo, días vencimiento)
 2. Implementar notificaciones automáticas de vencimiento
 3. Agregar validación de montos y fechas
 4. Mejorar método `createItems()` con validación
 5. Implementar generación automática de facturas recurrentes
 6. Agregar filtro por rangos de fecha en scopes
-

Última actualización: 12 de enero de 2026

Versión Laravel: 12

Desarrollador: Talentus Team